TECNICATURA UNIVERSITARIA
EN PROGRAMACIÓN
A DISTANCIA

Trabajo Práctico Semana 3: Automatización de Calidad y Blindaje

Objetivo: Automatizar la auditoría de calidad mediante *tooling* profesional y generar evidencias inmutables para la integración.

1. Configuración de un Pre-commit Hook: Configura un script local de *pre-commit hook* que ejecute de forma secuencial y obligatoria: Formatter, Linter y la suite de pruebas unitarias. Explica qué ocurre específicamente en la consola si uno de estos controles detecta un fallo y por qué esto protege la rama principal.

- Un script de *pre-commit hook* es una herramienta automatizada de Git que se dispara localmente al invocar el comando para guardar los cambios, antes de que el mensaje de confirmación sea consolidado en el historial.
- La cadena de custodia de calidad se ejecuta de forma secuencial: primero el Formatter para garantizar la estética visual, luego el Linter para auditar malas prácticas o variables en desuso, y finalmente la suite de Pruebas Unitarias.
- Si alguno de estos controles falla, el *hook* interrumpe la operación de forma abrupta, aborta el *commit* y devuelve un código de error visible en la consola.
- Este mecanismo protege la rama principal y el repositorio porque el código defectuoso queda retenido en el espacio de trabajo local del programador, forzándolo a sanear la deuda técnica antes de poder proponer el cambio.

2. Auditoría Estática: A partir de un código fuente que presenta baja calidad (duplicaciones, alta complejidad ciclomática y dependencias desactualizadas), utiliza las métricas de análisis estático para identificar los puntos críticos. Propón una estrategia de refactorización que permita reducir la complejidad y asegurar la seguridad de las dependencias.

El análisis estático con herramientas como SonarQube examina la salud estructural de la aplicación sin necesidad de ejecutar el software.

Ante una alta complejidad ciclomática, la cual indica un exceso de caminos lógicos independientes y bloques condicionales anidados, la estrategia de refactorización exige reducir estos flujos para mitigar la probabilidad de fallos y el costo de mantenimiento.

Cuando se detecta duplicación de código, la refactorización debe forzar la modularización unificando bloques idénticos para respetar el principio de no repetición (DRY).

Para garantizar la seguridad de las dependencias, las herramientas escanean archivos de manifiesto (como package.json o requirements.txt) contrastando las bibliotecas contra bases de datos globales de vulnerabilidades (CVE).

Si se encuentra una dependencia desactualizada o insegura, se debe aplicar la actualización hacia la versión estable y parchada recomendada por el reporte técnico.

3. Informe de Calidad en Verde: Consolida todas las evidencias generadas en los trabajos prácticos anteriores en un único *Informe de Calidad en Verde*. Tu informe debe incluir: el enlace trazable entre el Pull Request y el Issue, los logs de ejecución del pre-commit hook, el reporte analítico de cobertura de código y el diagnóstico final de análisis estático, certificando así que el desarrollo es apto para su integración.

El Informe de Calidad en Verde es el artefacto estructurado que certifica que el incremento de software cumple con la *Definition of Done* y está listo para integrarse. Se conforma de cuatro evidencias inmutables:

- Evidencia de Gobernanza de Entrada: El documento incluye el enlace trazable que vincula de manera explícita el Pull Request con el Issue o requerimiento original que motivó la tarea.
- Evidencia Sintáctica y Estética: Se adjuntan los registros de consola (logs) generados por el *pre-commit hook* demostrando que tanto el Formatter como el Linter se ejecutaron de manera exitosa y sin arrojar excepciones.
- Evidencia Dinámica: Se integra el reporte analítico de Cobertura de Código (Code Coverage) en estado verde, certificando matemáticamente que se superó el umbral mínimo exigido de pruebas locales.
- Evidencia Estructural: Se provee el diagnóstico final de Análisis Estático confirmando una deuda técnica aceptable, una complejidad ciclomática controlada y un escaneo que avala la ausencia de vulnerabilidades críticas en las dependencias.